Beyond Left-to-Right: A Comparative Study of Diffusion and Auto-Regressive Models for Code Infilling

Shivam Mittal UW Madison smittal39@wisc.edu

Gaurav Batra UW Madison gbatra3@wisc.edu

Grant Waldow UW Madison gwaldow@wisc.edu

Sujan Reddy UW Madison sujanreddy@cs.wisc.edu

Arshad Aslam Kazi UW Madison aakazi@wisc.edu

Abstract

We present a controlled empirical study comparing a diffusion-based language model (`Open-dCoder-0.5B`) (Fred Zhangzhi Peng and contributors, 2025) with auto-regressive (AR) counterparts of comparable and larger sizes (`Qwen2.5-[0.5/1.5]B-Instruct` (Team, 2024), `DeepSeek-Coder 1.3B`, `StarCoder2 3B`) on code infilling tasks. Diffusion language models (dLLMs) generate text through iterative denoising, enabling bidirectional reasoning which is absent from the monotonic left-to-right inference of standard AR models. We evaluate these models on `HumanEval-Infill` (Bavarian et al., 2022) and `SantaCoder-FIM` (Allal et al., 2023a), analyzing the trade-offs between functional correctness and exact token matching. Our results show that the 0.5B diffusion model achieves a **76.48% Pass@1** on HumanEval-Infill, outperforming the similarly sized Qwen2.5-0.5B (74.15%) and remaining competitive with models 6x its size (StarCoder2 3B). While dLLMs struggle with exact match metrics (55.99% vs. 64.91% for AR), our findings suggest that their bidirectional context modeling leads to superior functional correctness in code infilling tasks. Further, we present a heuristic based ensemble model that combines diverse capabilities of diffusion models and AR models. This models outperforms even the 3B model on the benchmarks. The project repository can be found at `https://github.com/msritian/Diffusion-Language-Models.git`

1 Introduction

Large language models currently being used for code generation are typically auto-regressive (AR), predicting tokens left-to-right (Chen et al., 2024). This means that an AR model can never go back and correct a previous token mistake (Li et al., 2025). This has been hypothesized to be the source of many undesirable LLM behaviors, such as hallucination and repeating(Bavarian et al., 2023). This paradigm may limit their performance on tasks that require bidirectional context, such as code infilling. Diffusion-based LLMs (dLLMs) have recently emerged as a promising alternative to AR models due to their efficient inference (Lei et al., 2025). Our goal is to evaluate whether diffusion models offer practical advantages over AR models for real-world code infilling tasks.

Contributions

- A controlled head-to-head comparison between a diffusion model and its auto-regressive base model.
- A quantitative benchmark analysis on two standard code infilling datasets.
- A qualitative analysis highlighting behavioral differences.
- Novel heuristic to combine Diffusion LLMs with Autoregressive LLMs

2 Background

2.1 Auto-Regressive Models for Code

Standard code LLMs model the probability of a token sequence $x = (x_1, ..., x_n)$ as the product of conditional probabilities: $p(x) = \prod_{i=1}^{n} p(x_i|x_{<i})$. For infilling tasks (Fill-In-the-Middle or FIM), this requires shuffling the data into a format like `<PRE> prefix <SUF> suffix <MID>`, effectively forcing the bidirectional task into a left-to-right format. While effective, this does not allow the model to revisit the "middle" generation based on constraints that might only become apparent later in the suffix. (Chen et al., 2021) (Fried et al., 2023)

2.2 Diffusion Language Models

Diffusion Language Models (dLLMs) adapt the diffusion probabilistic framework, popularized in image generation to discrete text (Sahoo et al., 2024). `Open-dCoder` specifically utilizes Masked Discrete

Diffusion (Zhang and collaborators, 2025). During training, tokens are randomly masked, and the model learns to predict the original tokens given the masked sequence. During inference, the model starts with a fully masked sequence and iteratively denoises it over T steps (e.g., 64 steps). Unlike AR models, the dLLM sees the embeddings of the prefix, the suffix, and its own evolving draft of the middle code simultaneously at every step, enabling true bidirectional reasoning (Lou et al., 2024).

2.3 Code Infilling as a Task

Code infilling is distinct from completion. The model must generate a code segment M that logically connects a prefix P and a suffix S. This is often harder than completion because M must adhere to variable naming conventions, return types, and logic flows dictated by S. It is a standard proxy for "refactoring" or "bug fixing" capability in developer tools (Fried et al., 2023).

2.4 Ensemble Modelling

Combining diverse models can help combine diverse learned hypotheses into a single, more sophisticated hypothesis (Reddy et al., 2022). In this context, diffusion LLMs have bidirectional context, so there are certain functions that it can learn well. On the other hand, AR models have more signal during pre-training due to more gradients from the Next token objective, resulting in a better model that can adapt to the data. So enseble modelling is a good application for this use case. (Huang et al., 2024) (Jiang et al., 2025)

3 Our Benchmarks

We employ two standard benchmarks to capture different aspects of generation quality.

3.1 HumanEval-Infill

`HumanEval-Infill` extends the OpenAI HumanEval benchmark to infilling tasks. It consists of 164 Python programming problems where parts of the solution (function body or signature) are masked. (OpenAI and contributors, 2022)

- **Metric: Pass@1**. This is an execution-based metric. The generated code is inserted into the problem harness and run against unit tests. It measures *functional correctness*, whether the code works—regardless of whether it matches the reference solution exactly.

3.2 SantaCoder-FIM

The `SantaCoder-FIM` benchmark, introduced by BigCode, evaluates the model's ability to reconstruct code from real-world repositories (GitHub). (Allal et al., 2023b)

- **Metric: Exact Match (EM)**. This compares the generated tokens directly to the ground truth reference code. It is a stricter metric that penalizes semantically correct code if it uses different variable names or formatting than the reference. It serves as a proxy for how well the model captures the specific distribution and style of the training data.

4 Experimental Setup

4.1 Models

We evaluate the `Open-dCoder-0.5B` model, a diffusion-based code generation model built on the `Open-dLLM` framework. For comparison, we select a suite of state-of-the-art auto-regressive models ranging from 0.5B to 3B parameters to provide both direct size comparisons and "punching up" comparisons.

For the ensemble model, we combine Open-dCoder-0.5B and Qwen2.5-Coder-0.5B-Instruct using a perplexity ranker. We sample responses from both these models and pick the one that has lower perplexity. This can help get the best of both worlds of diffusion LLMs and Autoregressive modelling. Model size wise, this is still smaller than the 3 larger language model we are using.

- **Diffusion:** `Open-dCoder-0.5B`
- **Auto-Regressive Baselines:**
  - `Qwen2.5-Coder-0.5B-Instruct` (Direct size equivalent)
  - `Qwen2.5-Coder-1.5B-Instruct`
  - `DeepSeek-Coder-1.3B`
  - `StarCoder2-3B`
- **Ensemble:** `Open-dCoder-0.5B` + `Qwen2.5-Coder-0.5B-Instruct`

5 Results

5.1 Quantitative Analysis

Our main results are summarized in Table 1. We observe distinct performance characteristics between diffusion and AR architectures.

Model	HumanEval-Infill (Pass@1)	SantaCoder-FIM (Exact Match)
Open-dCoder-0.5B	76.48	55.99
Qwen2.5-0.5B-Instruct	74.15	**64.91**
Qwen2.5-1.5B-Instruct	80.25	59.54
DeepSeek-Coder 1.3B	79.48	57.91
StarCoder2 3B	75.61	56.66
Ensemble 1B	**80.54**	60.69

Table 1: Comparison of Diffusion vs. Auto-Regressive models on code infilling. `Open-dCoder` outperforms its direct size competitor (Qwen 0.5B) on functional correctness (HumanEval) while scoring lower on Exact Match.

5.2 Discussion of Findings

Diffusion "Punches Above its Weight" on Logic. The most significant finding is that `Open-dCoder-0.5B` achieves a Pass@1 score of **76.48%** on HumanEval-Infill. This not only outperforms its direct size equivalent, `Qwen2.5-0.5B` (74.15%), but also surpasses `StarCoder2 3B` (75.61%), a model six times its size. This supports the hypothesis that the bidirectional context available during diffusion allows for superior logical consistency in code generation, even with fewer parameters.

The Exact Match Discrepancy. Conversely, the diffusion model lags significantly in Exact Match (EM) on SantaCoder-FIM (55.99% vs 64.91% for Qwen 0.5B). We hypothesize this is due to the nature of the diffusion generation process. AR models are trained to maximize the likelihood of the *exact* next token, incentivizing them to memorize specific syntactic patterns and variable names found in the training data. Diffusion models, by refining a noisy draft globally, may converge on a solution that is semantically equivalent (hence high HumanEval scores) but syntactically distinct from the reference solution (hence lower EM).

Scaling Laws Still Apply. While the 0.5B diffusion model performs exceptionally well for its size, larger AR models like `Qwen2.5-1.5B` (80.25%) still hold the absolute performance advantage. This suggests that while diffusion offers architectural efficiency gains, model scale remains a dominant factor in code reasoning capabilities.

Combining Hypotheses Works. Our ensemble method selects between the diffusion model (Open-dLLM) and the autoregressive model (Qwen) using a perplexity-based heuristic. This approach yields strong performance on HumanEval-Infill. The selector chose Open-dLLM in **19.94%** of cases (206/1033) and Qwen in **80.06%** (827/1033). The average perplexities reflect this distribution: Open-dLLM achieved an average perplexity of **3.93**, while Qwen obtained **2.14**. When compared directly, Open-dLLM had lower perplexity in **19.94%** of samples and Qwen in **39.79%**. Despite being selected less frequently, the diffusion model contributes meaningfully in cases requiring bidirectional context. Overall, the ensemble attains a **Pass@1 score of 80.54%**, outperforming all single-model baselines on HumanEval-Infill.

Table 2: Results for HumanEval-Infill (HE-Infill)

Metric	Open-dLLM	Qwen
Model selected (count)	206/1033	827/1033
Model selected (%)	19.94%	80.06%
Average Perplexity	3.93	2.14
Lower PPL (count)	206/1033	411/1033
Lower PPL (%)	19.94%	39.79%

Table 3: Results for Santacoder (SC)

Metric	Open-dLLM	Qwen
Model selected (count)	79/1043	964/1043
Model selected (%)	7.57%	92.43%
Average Perplexity	81,440.74	1.71
Lower PPL (count)	79/1043	920/1043
Lower PPL (%)	7.57%	88.21%

In contrast, the same complementary effect does not extend to the Santacoder benchmark. Out of 1043 samples, Open-dLLM was selected in only **7.57%** (79/1043), compared to **92.43%** for Qwen (964/1043). The perplexity gap is substantial: Open-dLLM averaged **81,440.74**, whereas Qwen achieved **1.71**. Qwen had lower perplexity in **88.21%** of cases, indicating minimal contribution from the diffusion model in this setting. Correspondingly, the ensemble reaches an Exact Match score of **60.69%**. These findings suggest that diffusion-based models provide clear benefits for *bidirectional infilling*, but offer limited advantage for standard left-to-right generation tasks.

6 Qualitative Error Analysis

To better understand the discrepancy between the high Pass@1 scores and lower Exact Match scores of `Open-dCoder`, we manually inspected generation samples. A recurring pattern emerged: the

diffusion model frequently generates functionally correct code that differs syntactically from the reference solution.

Figure 1 illustrates a representative example from the SantaCoder-FIM benchmark. The task is to fill in a missing condition check.

- **Reference (Ground Truth):** Checks if the variable `path` is `None`.
- **Open-dCoder (Diffusion):** Checks if `path` is `None`, but also includes an additional check for an empty string string (`or not path`).
- **Qwen2.5 (AR):** Reproduces the reference code exactly.

In this instance, the diffusion model's generation is semantically valid (and arguably more robust), but it is penalized by the Exact Match metric. This supports our hypothesis that the diffusion process, by modeling the "middle" based on the global context of the suffix, prioritizes satisfying the logical constraints of the surrounding code rather than predicting the exact next token probability of the training distribution. The ensemble just picks the version that is better.

```
# Context: Function to load a config file
def load_config(path):
    <FILL>
        return default_config()
    with open(path) as f:
        return yaml.safe_load(f)

# Reference Solution:
if path is None:

# Qwen2.5-0.5B (Auto-Regressive):
if path is None:

# Open-dCoder-0.5B (Diffusion):
if path is None or not path:
```

Figure 1: Comparison of generated infills. The diffusion model generates a more robust check that fails Exact Match.

7 Limitations

While promising, our study highlights several limitations of current diffusion language models:

1. **Inference Latency:** Although non-autoregressive, the iterative denoising process (64 steps in our experiments) is computationally expensive compared to a single forward pass of a small AR model. The latency for the ensemble is even higher because it involves 2 models.
2. **Exact Reproduction:** For tasks requiring strict adherence to a specific coding style or exact API usage (measured by Exact Match), AR models currently retain an edge.
3. **Context Length:** Our experiments were limited to relatively short context windows. The stability of diffusion generation over very long code contexts remains an open question.

8 Conclusion and Future Work

We presented a comprehensive evaluation of `Open-dCoder-0.5B` on code infilling tasks. Our results demonstrate that diffusion models can effectively compete with, and in some cases outperform, auto-regressive models of significantly larger size on functional correctness benchmarks. The ensemble model achieved a **80.54% Pass@1** on HumanEval-Infill, surpassing the 3B parameter StarCoder2.

We conclude that the bidirectional reasoning capability of diffusion models offers a tangible advantage for code infilling, allowing models to "look ahead" at the suffix to generate logically consistent code. Ensembling this with autoregressive models can help get the best of both worlds.

Future Work Future research should investigate:

- **Better Ensemble Architectures:** Experiment with other techniques to combine generations from the base models.
- **Task-Specific Tuning:** Fine-tuning diffusion models specifically for "bug fixing" tasks, where the bidirectional context is paramount.
- **Latency Optimization:** Exploring methods to reduce the number of denoising steps without compromising generation quality.

Code Availability

The source code for this project is publicly available at: `https://github.com/msritian/Diffusion-Language-Models.git`. Full experimental logs, including system metrics and generation samples, are available in our Weights & Biases report:

- Open-dcoder
- Benchmark for AR models
- Metrics for Emsemble models

References

Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Munoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, Logesh Kumar Umapathi, Carolyn Jane Anderson, Yangtian Zi, Joel Lamy Poirier, Hailey Schoelkopf, Sergey Troshin, Dmitry Abulkhanov, Manuel Romero, Michael Lappert, Francesco De Toni, Bernardo García del Río, Qian Liu, Shamik Bose, Urvashi Bhattacharyya, Terry Yue Zhuo, Ian Yu, Paulo Villegas, Marco Zocca, Sourab Mangrulkar, David Lansky, Huu Nguyen, Danish Contractor, Luis Villa, Jia Li, Dzmitry Bahdanau, Yacine Jernite, Sean Hughes, Daniel Fried, Arjun Guha, Harm de Vries, and Leandro von Werra. 2023a. Santacoder: don't reach for the stars!

Loubna Ben Allal, George Zerveas, Niklas Muennighoff, Denis Kocetkov, Raymond Li, Nouamane Tazi, and BigCode collaborators Anthropic. 2023b. Santacoder: Don't reach for the stars! *arXiv preprint arXiv:2301.03988*.

Mohammad Bavarian, Heewoo Jun, Nikolas Tezak, John Schulman, Christine McLeavey, Jerry Tworek, and Mark Chen. 2022. Efficient training of language models to fill in the middle. *arXiv preprint arXiv:2207.14255*.

Mohammad Bavarian, Augustus Odena, Tom Henighan, Mark Chen, Heewoo Chen, Carla Gomes, and Ilya Sutskever. 2023. Large language models of code fail at completing code with bugs. In *Advances in Neural Information Processing Systems*.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Yuchen Chen, Xinyun Li, Yiding Wang, Ming Xu, Jingbo Wang, and Denny Zhou. 2024. Diffusion language models for code infilling. In *The Twelfth International Conference on Learning Representations*.

Alex Tong Fred Zhangzhi Peng, Shuibai Zhang and contributors. 2025. Open-dllm: Open diffusion large language models. `https://github.com/pengzhangzhi/Open-dLLM`. Model: `https://huggingface.co/fredzzp/open-dcoder-0.5B`.

Daniel Fried, Armen Aghajanyan, Jessy Lin, Sida I. Wang, Luke Zettlemoyer, and Mike Lewis. 2023. Incoder: A generative model for code infilling and synthesis. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Yichong Huang, Xiaocheng Feng, Baohang Li, Yang Xiang, Hui Wang, Ting Liu, and Bing Qin. 2024. Ensemble learning for heterogeneous large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Dong Jiang, Xiang Ren, et al. 2025. Harnessing multiple large language models: A survey on llm ensemble. *arXiv preprint arXiv:2502.18036*.

Jiayi Lei, Chen Huang, Yuxuan Li, Shuo Wang, and Yue Zhang. 2025. Unlocking the potential of diffusion language models with template infilling. *arXiv preprint arXiv:2510.13870*.

Hao Li, Rui Zhang, Yuchen Wang, Yuchen Chen, Yizhou Sun, and Denny Zhou. 2025. Dreamon: Diffusion language models for code infilling beyond fixed-length generation. In *Advances in Neural Information Processing Systems*.

Aaron Lou, Francisco J. R. Ruiz, Nick Pawlowski, and Alexandre Lacoste. 2024. Simplified and generalized masked diffusion for discrete data. *arXiv preprint arXiv:2406.04329*.

OpenAI and contributors. 2022. Humaneval-infilling: A benchmark for code infilling derived from humaneval. Project and dataset documentation. Benchmark for code infilling tasks derived from the HumanEval dataset.

Sujan Reddy, S Akashdeep, R Harshvardhan, and Sowmya Kamath. 2022. Stacking deep learning and machine learning models for short-term energy consumption forecasting. *Advanced Engineering Informatics*, 52:101542.

Subham Sahoo, Jingyun Xu, Sidi Zhang, Xinyang Li, Yunzhi Chen, Yunzhe Li, and Volodymyr Kuleshov. 2024. Simple and effective masked diffusion language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Qwen Team. 2024. Qwen2.5: A party of foundation models.

Peng Zhang and collaborators. 2025. Open-dllm and open-dcoder: Masked diffusion language models for code generation. `https://github.com/pengzhangzhi/Open-dLLM`. Open diffusion language models and Open-dCoder implementation.